Week 9 In-Class Exercises (Extra) (while-loops)

Q1: Strings [**/***]

Define a function called `get_strings()` that takes in two parameters: (1) A list of strings called `str_list`. (2) An integer value `t`. The function returns a list that contains the **first** `n` strings in `str_list` such that ***the total number of characters*** in these n strings is **strictly** larger than `t`. The function should find the smallest `n` that satisfies the condition above and return those `n` strings. If the total number of digits in all the strings in `str_list` is still not larger than `t`, then the function returns a list that contains all the strings in `str_list`. You must use a while loop for this question.

For example, suppose `str_list` is `['a', 'bc', 'defgh', 'ij', 'k', 'lmn']`, and `t` is 3, then the function should return `['a', 'bc', 'defgh']`, because the number of digits in these three strings is 8, which is larger than 3.

Q2: Password [**/***]

Prompt the user for a password that he/she wants to choose. A valid password should satisfy the following condition:

a. It has to be at least 6-character long.
b. It cannot exceed 10 characters.
c. It cannot contain any space.
d. It has to contain at least a lowercase letter.
e. It has to contain at least an uppercase letter.
f. It has to contain at least a digit (0 to 9).
g. It has to contain at least one character that is not a letter or a digit.

Keep prompting the user until the password entered is valid.

Your code should provide the following output:

```
Enter a password: Hello
The password entered is not valid!
Enter a password: Hello_IS111
The password entered is not valid!
Enter a password: Hi guys!
The password entered is not valid!
Enter a password: Hi-IS111
Thank you!
```

Q3: Merge Lists [***]

Define a function called `merge_by_age()` that merges two lists of people. The function takes in two lists, where each list contains tuples representing people with their name and age information. **Each list is sorted in increasing order of the people's age.** For example, `list1` may look like `[("John", 12), ("Kate", 15), ("Henry", 35)]`, and `list2` may look like `[("Mike", 18), ("Scott", 20), ("Joseph", 48), ("Larry", 54)]`. The function merges the two lists into a single list in which the people are still ordered by their age. For example, `merge_by_age(list1, list2)` should return `[("John", 12), ("Kate", 15), ("Mike", 18), ("Scott", 20), ("Henry", 35), ("Joseph", 48), ("Larry", 54)]`.

Q4: Group Numbers [***]

Define a function called `group_numbers`. The function takes in two parameters: (1) A list of integers. (2) A threshold. The function tries to group the numbers in the list such that the sum of the numbers in each group does not exceed the specified threshold. The numbers should be grouped sequentially. The function should try to create as few groups as possible. The groups are returned as a list of lists.

For example, suppose the list of numbers given to the function is `[1, 3, 2, 4, 3, 2, 3, 6]`. Suppose the threshold is 6. The function will group the first three numbers, 1, 3 and 2, together, because their sum is exactly 6. The next number, 4, cannot be grouped with the subsequent number 3 because their sum would exceed 6.

`group_numbers([1, 3, 2, 4, 3, 2, 3, 6], 6)` should return `[ [1, 3, 2], [4], [3, 2], [3], [6] ]`. We can see that the first three numbers 1, 3 and 2 are grouped together because their sum is 6, which does not exceed the threshold 6. The next number, 4, cannot be grouped with the subsequent number, 3, because 4 + 3 has exceeded the threshold 6.

You can assume that none of the numbers in the list of integers is larger than the threshold. If the number list is empty, the function returns an empty list.

Q5: Binary Search [***/Out of scope]

Analyze the worst-case complexity of the function below, which implements a search algorithm called binary search.

```
def binary_search_iterative(arr, target):
    """
    Perform binary search on a sorted array to find the target value iteratively.

    Parameters:
    arr (list): A sorted list of elements.
    target: The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (right + left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Example usage
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
target = 7
index = binary_search_iterative(arr, target)
print(f"Index of {target}: {index}")
```

Note: logarithmic complexity analysis is out of scope and won't be tested.